École Nationale d'Ingénieurs de Sousse

Département Électronique Industriel

Rapport projet semestriel :

“DriveAlert”:
Système embarqué de détection de somnolence au volant

Projet réalisé par :

Sahar GHAZEL

Zeineb KCHOUK

Wiem ZAGHOUAN

Wissal ZANZOURI

Filière:

EI 2.3

Encadré par : M. Adel BOUALLEGUE

Année Universitaire :

2025-2026

Table de matières

I. Introduction : 6
II. Présentation du projet : 7
1. Contexte et problématique: 7
2. Objectifs SMART du projet : 7
3. Parties prenantes : 8
4. Périmètre de projet : 8
5. Analyse fonctionnelle : 9
5.1. Fonctions Principales du Système : 9
5.2. Exigences Techniques : 9
5.3 Exigences Non Fonctionnelles : 10
III. Management du projet : 11
1. Introduction : 11
2. Méthode de management appliquée : 11
3. Structure de Découpage du Projet (WBS): 11
4. Planning Détaillé : Répartition des Tâches par Étudiante : 13
5. Diagramme de Gantt : 14
6. Budget Estimatif : 15
7. Analyse et gestion des risques : 16
7.1. Registre des Risques : 16
7.2. Plan d'Action pour les Risques Critiques (Criticité ≥ 6) : 18
8. Exécution et suivi du projet: 21
8.1. Introduction: 21
8.2. Référentiel de Management Appliqué (PMP/PMI) : 21
8.3. Matrice RACI du Projet : 23
9. Validation et Clôture du Projet : 25
9.1. Introduction : 25
9.2. Plan de Tests : 25
9.3. Jalons de Validation (Gate Reviews) : 26
9.4. Livrables finaux : 26
9.5. Critères de Succès Global : 27
IV. Conception et Choix Technologiques : 28
1. Introduction : 28
2. Sélection des composants : 28
2.1. Microcontrôleur : 28
2.2 Module Caméra : Raspberry Pi Camera NoIR v2 : 29
2.3 Composants de détection et de signalisation: 30
2.4 Outils de programmation et d'intégration: 31
3. Le modèle d'intelligence artificielle 32
3.1 La base de données — MRL Eye Dataset (Kaggle) 33

3.2 Choix de l'architecture CNN et justification..... 34
3.3 Entraînement du modèle.....35
3.4. Conversion et export en TFLite.....37
3.5. Algorithme hybride EAR + CNN.....38
4. Design électronique et câblage:..... 38
5. Conclusion..... 40
V. Conclusion générale.....41
VI. Bibliographie.....42

Table de figures

Figure 1: Courbes d'apprentissage……...35

Figure 2: Matrice de confusion ……...36

I. Introduction :

La sécurité routière représente aujourd'hui un défi majeur pour les sociétés modernes, en raison du nombre croissant d'accidents de la circulation et de leurs conséquences humaines, sociales et économiques. Malgré l'amélioration continue des infrastructures routières et l'évolution des systèmes de sécurité intégrés aux véhicules, de nombreux accidents restent causés par des facteurs humains tels que l'inattention, la distraction, le stress et surtout la fatigue. Parmi ces facteurs, la somnolence au volant constitue un danger particulièrement critique, car elle agit souvent de manière progressive et silencieuse, réduisant la vigilance du conducteur sans qu'il en prenne pleinement conscience.

Lorsqu'un conducteur est en état de fatigue avancée, ses capacités de concentration diminuent, ses réflexes deviennent plus lents et son temps de réaction face aux imprévus augmente considérablement. Dans certains cas, des épisodes de micro-sommeil peuvent survenir pendant quelques secondes seulement, mais ces instants suffisent pour perdre le contrôle du véhicule et provoquer des accidents parfois mortels. Ce phénomène touche particulièrement les conducteurs effectuant de longs trajets, les professionnels du transport, les personnes circulant de nuit ou encore celles soumises à un rythme de vie intense.

Face à cette problématique, les méthodes traditionnelles de prévention, telles que les campagnes de sensibilisation ou les pauses recommandées durant la conduite, demeurent importantes mais parfois insuffisantes. L'intégration de solutions technologiques intelligentes dans les véhicules apparaît alors comme une approche innovante et efficace pour assister le conducteur en temps réel. Grâce aux avancées récentes en électronique embarquée, en intelligence artificielle et en vision par ordinateur, il devient désormais possible de surveiller certains indicateurs physiques révélateurs de fatigue, notamment l'état des yeux et la fréquence de clignement.

II. Présentation du projet :

1. Contexte et problématique:

La somnolence au volant constitue l'une des premières causes d'accidents mortels en Tunisie et dans le monde. Selon les données de l'Observatoire National de la Sécurité Routière, plus de 20 % des accidents graves sur autoroute sont liés à la fatigue du conducteur. Ce constat soulève un enjeu de sécurité publique majeur, pour lequel les solutions technologiques embarquées et accessibles représentent une réponse prometteuse.

Dans le cadre de ce projet semestriel à l'ENISO, notre groupe de quatre étudiantes en Génie Électronique Industrielle propose de concevoir et de déployer un système de détection de somnolence en temps réel. Ce système repose sur un Raspberry Pi 4, une caméra infrarouge, et un modèle de deep learning (réseau de neurones convolutif CNN) entraîné sur le MRL Eye Dataset, combiné à un algorithme géométrique de calcul de l'Eye Aspect Ratio (EAR).

2. Objectifs SMART du projet :

Les objectifs sont structurés selon la méthode SMART (Spécifique, Mesurable, Atteignable, Réaliste, Temporellement défini) :

ID	Description	Critère de succès	Priorité
Spécifique	Détecter la somnolence avec une précision > 95%	Accuracy > 95% sur le test set MRL	**OBLIGATOIRE**
Mesurable	Alerter le conducteur en < 1 seconde	Délai mesuré < 1s en conditions réelles	**OBLIGATOIRE**
Réaliste	Minimiser les faux négatifs (somnolence non détectée)	Recall classe sleepy > 97%	**CRITIQUE**
Temporellement défini	Livrer le projet avant le 4 mai 2026	Soutenance et démonstration validées	**OBLIGATOIRE**

3. Parties prenantes :

Partie prenante	Rôle	Attentes
Groupe projet (4 étudiantes)	Réalisation technique complète	Valider toutes les phases dans les délais
Encadrant pédagogique ENISo	Supervision et évaluation	Respect du CDC, qualité scientifique et technique
Jury de soutenance	Évaluation finale	Démonstration fonctionnelle + rapport complet
Utilisateurs finaux (conducteurs)	Bénéficiaires du système	Fiabilité, réactivité, absence de fausses alertes critiques

4. Périmètre de projet :

Inclus dans le périmètre :

- Préparation du MRL Eye Dataset (split train/val/test, augmentation de données)
- Développement et entraînement du modèle CNN sur Google Colab (GPU T4)
- Conversion du modèle en TFLite (quantification float16)
- Intégration embarquée sur Raspberry Pi 4 : caméra, buzzer GPIO
- Algorithme hybride EAR (Eye Aspect Ratio) + CNN pour la décision
- Tests de validation et rapport de projet complet

Hors périmètre :

- Application mobile ou interface web de télésurveillance
- Intégration dans un véhicule réel (OBD-II, CAN Bus)
- Certification ou homologation réglementaire

5. Analyse fonctionnelle :

5.1. Fonctions Principales du Système :

ID	Fonction	Description	Priorité
FP1	Acquisition vidéo	Capturer le visage du conducteur en continu (30 FPS, 640×480)	**Obligatoire**
FP2	Détection du visage	Localiser les 468 points faciaux en temps réel via MediaPipe	**Obligatoire**
FP3	Calcul EAR	Calculer l'Eye Aspect Ratio sur 6 landmarks par œil (scipy)	**Obligatoire**
FP4	Classification CNN	Prédire ouverture/fermeture des yeux via modèle TFLite embarqué	**Obligatoire**
FP5	Décision hybride	Combiner EAR et CNN (logique ET) sur une fenêtre de 10 frames	**Obligatoire**
FP6	Alerte physique	Activer buzzer (GPIO 18) en cas de somnolence	**Obligatoire**
FP7	Affichage temps réel	Afficher EAR, score CNN, compteur et état sur l'écran	Souhaitable

5.2. Exigences Techniques :

Réf.	Exigence	Valeur cible	Priorité
ET-01	Précision globale du modèle CNN sur test set	> 95 %	**P1**
ET-02	Recall classe sleepy (faux négatifs minimaux)	> 97 %	**P1**
ET-03	Cadence de traitement vidéo sur Raspberry Pi 4	> 20 FPS	**P1**
ET-04	Délai total détection → déclenchement alerte	< 1 seconde	**P1**
ET-05	Taille du modèle TFLite exporté	< 2 MB	P2

ET-06	Consommation mémoire RAM (Raspberry Pi)	< 512 MB	P2
ET-07	Fonctionnement autonome hors réseau	100 % embarqué	**P1**
ET-08	Tolérance aux variations de luminosité	100 – 800 lux	P2

5.3 Exigences Non Fonctionnelles :

Catégorie	**Exigence**	**Critère d'acceptation**
Fiabilité	Fonctionnement continu sans crash	Aucun arrêt intempestif sur 2 heures de test
Performance	Inférence CNN rapide sur CPU	< 50 ms par inférence sur Pi 4
Maintenabilité	Code source commenté et structuré	Compréhensible par un ingénieur
Sécurité données	Aucun stockage du visage du conducteur	Traitement local en mémoire uniquement
Reproductibilité	Pipeline d'entraînement documenté	Résultats reproduits avec même dataset

III. Management du projet :

1. Introduction :

La planification est le cœur du management de projet. Elle permet de transformer les objectifs en un plan d'action concret, répartissant les tâches, les délais, les responsabilités et les ressources. Ce chapitre présente tous les outils de planification que nous avons utilisés.

2. Méthode de management appliquée :

Bien que notre projet soit académique, nous avons adopté les principes fondamentaux du management de projet selon le référentiel PMP (Project Management Professional, PMI), adaptés à notre contexte de 5 semaines.

3. Structure de Découpage du Projet (WBS):

Le projet est découpé en 5 phases adaptées à la durée semestrielle contrainte de 35 jours. Chaque phase correspond à une semaine de travail intensive, avec des livrables intermédiaires validés en groupe.

Phase	Intitulé	Période	Tâches principales
Phase 1	Analyse et Planification	Jours 1–5	– Définition des objectifs et périmètre du projet – Étude bibliographique (somnolence, EAR, CNN, MRL Dataset) – Choix de l'architecture matérielle (Raspberry Pi 4, caméra) – Identification des risques initiaux – Rédaction du cahier des charges
Phase 2	Collecte et Préparation des Données	Jours 6–12	– Téléchargement du MRL Eye Dataset (archive.zip) – Exploration et validation de la structure du dataset – Organisation des splits (train / val / test) – Analyse des classes (awake=0 / sleepy=1) – Définition de la stratégie d'augmentation de données

Phase 3	Modélisation et Entraînement CNN	Jours 13–22	– Configuration de l'environnement Google Colab (GPU T4) – Implémentation de l'architecture CNN (3 blocs Conv2D) – Mise en place des callbacks (EarlyStopping, ReduceLR, Checkpoint) – Entraînement du modèle (max 30 epochs) – Analyse des courbes d'apprentissage (accuracy / loss) – Évaluation sur le test set — objectif > 95% – Conversion du modèle en TFLite (quantification Float16)
Phase 4	Intégration Raspberry Pi	Jours 23–30	– Configuration du Raspberry Pi 4 (OS, dépendances) – Câblage GPIO (buzzer PIN 18, LED PIN 24) – Intégration MediaPipe Face Mesh (landmarks yeux) – Calcul EAR (Eye Aspect Ratio) en temps réel – Inférence TFLite (toutes les 3 frames) – Logique de détection combinée EAR + CNN – Tests unitaires et validation sur le terrain
Phase 5	Tests, Documentation et Rapport	Jours 31–35	– Tests de performance (latence, FPS, consommation CPU) – Validation des seuils (EAR=0.20, CNN=60%, Frames=15) – Rédaction du rapport final – Préparation de la soutenance et démonstration – Livraison des livrables (code, modèle, documentation)

Jalons clés du projet :

- J5 — Jalon J5 : Cahier des charges validé
- J12 — Jalon J12 : Dataset structuré et validé

- J22 — Jalon J22 : Modèle CNN entraîné > 95% accuracy + TFLite exporté
- J30 — Jalon J30 : Intégration complète et tests terrain validés
- J35 — Jalon J35 : Rapport final remis + soutenance

4. Planning Détaillé : Répartition des Tâches par Étudiante :

Chaque étudiante se voit attribuer un domaine de responsabilité principal. La collaboration est assurée lors des phases de test et de rédaction du rapport.

Étudiante	Domaine de responsabilité	Tâches assignées
Sahar	Données & Modèle CNN	– Téléchargement et organisation du MRL Eye Dataset – Développement du pipeline d'augmentation de données – Implémentation de l'architecture CNN (train_mrl_dataset.py) – Suivi des courbes d'apprentissage et ajustement des hyperparamètres – Évaluation finale : matrice de confusion, rapport de classification
Zeineb	Intégration Raspberry Pi & GPIO	– Configuration du Raspberry Pi 4 et installation des dépendances – Câblage et programmation GPIO (buzzer + LED) – Intégration du modèle TFLite (tflite_runtime) – Implémentation de la logique de détection combinée EAR + CNN – Optimisation des performances (inférence toutes les 3 frames)
Wiem	Vision & Interface Temps Réel	– Intégration MediaPipe Face Mesh et extraction des landmarks – Calcul de l'Eye Aspect Ratio (EAR) en temps réel – Développement de l'interface d'affichage (dessiner_interface) – Tests de validation sur le terrain (seuils EAR, CNN, frames) – Rédaction de la documentation technique et du rapport final

Wissal	Tests, Qualité & Management	– Rédaction du cahier des charges et suivi du planning (Gantt) – Coordination entre les membres et gestion des risques projet – Tests d'intégration système complets (EAR + CNN + GPIO) – Validation des seuils de détection et analyse des faux négatifs – Préparation de la soutenance et conception du poster

5. Diagramme de Gantt :

Le planning prévisionnel est organisé sur 5 semaines (35 jours). Les barres pleines indiquent l'activité prévue sur chaque semaine. La colonne Resp. indique la ou les étudiantes responsables (A, B, C).

Tâche / Livrable	**S1 J1–7**	**S2 J8–14**	**S3 J15–21**	**S4 J22–28**	**S5 J29–35**	**Resp.**
Analyse et Planification						**Sahar/Wissal**
Collecte et Préparation données						**Sahar**
Analyse et Planification						**Sahar/Wissal**
Collecte et Préparation données						**Sahar**
Config Colab et CNN Build						**Sahar**
Entraînement et Courbes						**Sahar**
Évaluation et Export TFLite						**Sahar**
Config Pi et GPIO						**Zeineb**
MediaPipe et Calcul EAR						**Wiem**
Inférence TFLite + Logique						**Zeineb/Wiem**

Tests unitaires						**Wissal**
Tests perf et Validation seuils						**Sahar/Zeineb/Wiem/Wissal**
Rapport et Soutenance						**Sahar/Zeineb/Wiem/Wissal**

Chemin critique

Le chemin critique passe par :

Préparation données (J6-J12) → Entraînement CNN (J13-J22) → Intégration Pi (J23-J30).

Tout retard sur l'entraînement CNN décale directement l'intégration et les tests finaux.

6. Budget Estimatif :

Le budget total estimé est de 210,76 € HT. Ce projet est essentiellement basé sur des ressources open-source (Kaggle, Google Colab), ce qui minimise les coûts logiciels.

Poste / Article	Qté	Prix unitaire	Total HT
Matériel Informatique			
Raspberry Pi 4 Model B (4 Go RAM)	1	530 TND	530 TND
Module caméra Raspberry Pi (5 Mpx)	1	31 TND	31 TND
Carte microSD 32 Go (Class 10)	1	22 TND	22 TND
Alimentation officielle Pi 4 (USB-C 3A)	1	38.5 TDN	38.5 TDN
Câbles, breadboard, résistances	1	25 TND	25 TDN
Composants Électroniques			
Buzzer actif 5V	1	1,349 TND	1,349 TND
LED rouge haute luminosité	2	0,100 DT	0,200 DT
Résistances 220 Ω (lot)	1	0,100 DT	0,100 DT
Ressources Cloud & Logicielles			
Google Colab (gratuit)	1	0,00 TDN	0,00 TDN

MRL Eye Dataset (Kaggle — gratuit)	1	0,00 TDN	0,00 TDN
		TOTAL ESTIMATIF DU PROJET	**648.149 TND**

Analyse des coûts

Le poste le plus significatif est le matériel Raspberry Pi (135 €, soit 64% du budget). Les ressources logicielles sont quasi-nulles grâce aux outils open-source. La réserve de 10% (19,16 €) couvre les composants de remplacement et frais imprévus.

7. Analyse et gestion des risques :

L'analyse des risques s'appuie sur une matrice Probabilité × Impact (échelle 1-5). La criticité est le produit P×I. Les risques de criticité ≥ 6 font l'objet d'un plan d'action détaillé (section 7.2).

Criticité	Niveau	Action requise
≥ 9	**CRITIQUE**	Plan d'action immédiat obligatoire
6 – 8	**HAUTE**	Plan d'action requis (section 7.2)
4 – 5	**MOYENNE**	Surveillance et action préventive
1 – 3	**FAIBLE**	Accepter / Surveiller

7.1. Registre des Risques :

P = Probabilité (1=Rare → 5=Quasi-certain) | I = Impact (1=Négligeable → 5=Catastrophique) | C = Criticité = P × I

ID	Risque identifié	Catégorie	P	I	C=P×I	Resp.	Priorité	Action préventive
R01	Surapprentissage (Overfitting) du CNN	Technique	3	4	**12**	**Sahar**	**HAUTE**	EarlyStopping, Dropout 0.25/0.5, Augmentation données

R02	Modèle TFLite incompatible avec tflite-runtime	Technique	2	5	**10**	**Sahar/ Zeineb**	**HAUTE**	Test de cohérence (Cellule 11) avant déploiement Pi
R03	Latence trop élevée sur Raspberry Pi (< 10 FPS)	Performance	3	4	**12**	**Zeineb**	**HAUTE**	Inférence toutes les 3 frames, Float16, IMG_SIZE=64
R04	Faux négatifs (somnolence non détectée)	Sécurité	2	5	**10**	**Wissal**	**CRITIQUE**	Ajuster SEUIL_EAR / SEUIL_MODELE / FRAMES_AVANT_ALERTE
R05	Faux positifs excessifs (alarmes intempestives)	Fonctionnel	3	3	**9**	**Wiem**	**MOYENNE**	Seuil EAR+0.03 pour désactivation, validation terrain
R06	Détection visage échouée (éclairage faible)	Environnement	3	4	**12**	**Wiem**	**HAUTE**	Min detection confidence 0.5, éclairage IR optionnel
R07	Panne du buzzer / LED GPIO	Matériel	2	3	**6**	**Zeineb**	**FAIBLE**	Composants de rechange, test GPIO.output avant intégration
R08	Dataset non représentatif (ethnies, éclairages)	Données	2	4	**8**	**Sahar**	**MOYENNE**	Augmentation (luminosité, zoom, flip), MRL multi-conditions
R09	Retard dans l'entraînement Colab (quota GPU)	Ressources	2	3	**6**	**Sahar**	**FAIBLE**	Colab Pro, sauvegardes ModelCheckpoint, sessions alternatives
R10	Conflits de versions bibliothèques Python	Technique	2	3	**6**	**Wissal**	**FAIBLE**	Utiliser requirements.txt figé, environnement virtuel

7.2. Plan d'Action pour les Risques Critiques (Criticité ≥ 6) :

Les 4 risques suivants ont une criticité ≥ 6 et nécessitent un plan d'action formalisé. Ils sont classés par ordre de priorité décroissante.

- Risque R04 — Faux Négatifs : Somnolence Non Détectée — Criticité : 10 (P=2 × I=5)

Situation :

Le système n'alerte pas alors que le conducteur est endormi. C'est le risque le plus dangereux du projet.

Causes identifiées :

- Seuil SEUIL_EAR trop élevé (0.20 insuffisant pour certains visages)
- SEUIL_MODELE trop bas (0.60) — CNN peu certain avant d'alarmer
- Nombre de frames insuffisant (FRAMES_AVANT_ALERTE=15 trop permissif)
- Mauvaise extraction de la région de l'œil (landmark imprécis)

Plan d'action :

Action corrective / préventive	Qui	Quand	Modalités / Critère de succès
Réduire SEUIL_EAR à 0.18	Wissal	Phase 5	Tests terrain avec plusieurs conducteurs, mesure EAR de base par personne
Augmenter SEUIL_MODELE à 0.55	Sahar	Phase 5	Réévaluation sur test set — vérifier que le taux fausse alarme reste acceptable
Réduire FRAMES_AVANT_ALERTE à 12	Zeineb	Phase 4	Alerte plus rapide (~0.4 sec à 30fps) tout en évitant les faux positifs
Validation croisée avec 3 conditions d'éclairage	Wiem	Phase 5	Tests en intérieur jour, nuit, contre-jour

- Risque R02 — Incompatibilité Modèle TFLite — Criticité : 10 (P=2 × I=5)

Situation :

Le modèle converti en TFLite produit des prédictions incohérentes ou plante sur le Raspberry Pi.

Causes identifiées :

- Version incompatible entre TensorFlow (Colab) et tflite-runtime (Pi)
- Quantification Float16 mal supportée par l'architecture cible
- Mauvais prétraitement (normalisation différente du training)

Plan d'action :

Action corrective / préventive	Qui	Quand	Modalités / Critère de succès
Test de cohérence systématique (Cellule 11)	Sahar	Fin Phase 3	Comparer prédictions TFLite vs Keras sur 20 images avant déploiement
Figer les versions dans requirements.txt	Zeineb	Phase 4	tflite-runtime==2.12.0, opencv==4.8.0, mediapipe==0.10.x
Fallback : garder meilleur_modele.keras + tf.lite.Interpreter	Sahar/Zeineb	Phase 4	Alternative si tflite-runtime pose problème

- Risque R01/R03 — Surapprentissage & Latence Pi — Criticité : 12 (P=3 × I=4)

Situation :

Le CNN overfit sur le dataset ou tourne trop lentement (<10 FPS) sur le Pi, rendant la détection non fiable.

Causes identifiées :

- Dataset déséquilibré (plus d'images awake que sleepy)
- Dropout insuffisant ou absence de régularisation
- Inférence CNN trop fréquente (chaque frame = CPU saturé)

Plan d'action :

Action corrective / préventive	Qui	Quand	Modalités / Critère de succès

Surveiller val_accuracy vs train_accuracy	Sahar	Phase 3	Arrêt si écart > 5% → augmenter Dropout ou réduire modèle
Inférence CNN toutes les 3 frames (frame_count % 3)	Zeineb	Phase 4	Déjà implémenté dans detection_pi_mrl.py — valider à 30 FPS
Réduire IMG_SIZE à 48 si FPS < 10	Sahar/Zeineb	Phase 4	Retraining rapide avec IMG_SIZE=48 — impact précision < 1%

- Risque R06 — Échec Détection Visage (Éclairage) — Criticité : 12 (P=3 × I=4)

Situation :

MediaPipe Face Mesh ne détecte pas le visage dans des conditions d'éclairage difficiles (nuit, contre-jour).

Causes identifiées :

- min_detection_confidence=0.5 trop restrictif en faible luminosité
- Absence d'éclairage infrarouge pour la conduite nocturne
- Arrière-plan complexe perturbant la détection

Plan d'action :

Action corrective / préventive	Qui	Quand	Modalités / Critère de succès
Afficher message d'avertissement 'Aucun visage détecté'	Wiem	Phase 4	Déjà implémenté — ajouter compteur : si > 30 frames sans visage → alerte
Tester avec min_detection_confidence=0.3 en mode nuit	Wiem	Phase 5	Balance entre détection et faux positifs
Recommander LEDs IR externes (budget réserve)	Zeineb	Phase 5	Module IR 850nm compatible Pi — ~15€ inclus dans réserve

8. Exécution et suivi du projet:

8.1. Introduction:

L'exécution est la phase centrale du cycle de vie d'un projet : c'est là que les plans théoriques se confrontent à la réalité opérationnelle. Pour notre projet de système embarqué de détection de somnolence au volant, cette phase couvre les semaines 2 à 4 (du 6 au 25 avril 2026), soit la période de développement intensif allant de l'entraînement du modèle CNN jusqu'à l'intégration complète sur Raspberry Pi 4.

Le suivi du projet a été assuré selon deux axes complémentaires :

- Un suivi opérationnel hebdomadaire : chaque membre a rendu compte de l'avancement de ses tâches lors des réunions du vendredi (Gate Reviews G1 à G5).
- Un suivi par tableau de bord : un indicateur d'avancement (%) a été maintenu pour chacune des 5 phases du WBS, permettant une détection précoce des écarts.

Ce chapitre présente le référentiel de management appliqué (PMP/PMI), la matrice RACI formalisée, la répartition détaillée des tâches par membre, le suivi de l'avancement, les décisions prises en cours d'exécution, ainsi que les défis rencontrés et les solutions apportées.

8.2. Référentiel de Management Appliqué (PMP/PMI) :

Bien que ce projet soit de nature académique et d'une durée limitée à 5 semaines, nous avons délibérément choisi d'appliquer les principes du référentiel PMP (Project Management Professional) du PMI (Project Management Institute). Ce choix pédagogique vise à ancrer notre démarche dans des pratiques professionnelles reconnues à l'échelle internationale.

Le PMBOK® Guide (Project Management Body of Knowledge) structure la gestion de projet autour de 10 domaines de connaissance. Le tableau ci-dessous présente la correspondance entre ces domaines et les actions concrètes mises en œuvre dans notre projet :

Domaine PMP	Application concrète dans notre projet
Intégration	Charte de projet validée collectivement + réunion de lancement le 30 mars 2025. Pilotage centralisé par la chef de projet Zeineb.

Périmètre (Scope)	WBS défini en 5 phases. Gestion des demandes hors périmètre formalisée. Exclusions clairement documentées dans le cahier des charges.
Délais	Planning Gantt sur 5 semaines, jalons hebdomadaires (G1 à G5), date de soutenance fixe le 4 mai 2025.
Coûts	Budget matériel estimé à 113 €. Utilisation exclusive de logiciels et outils open source (Colab, Kaggle, OpenCV, TFLite).
Qualité	Tests unitaires et d'intégration (T-01 à T-08), métriques ML : accuracy ≥ 95 %, recall sleepy ≥ 97 %.
Ressources humaines	4 étudiantes avec rôles définis et complémentaires. Matrice RACI formalisée pour chaque activité du projet.
Communications	Point hebdomadaire en groupe, partage de documents via Google Drive et GitHub, canal WhatsApp de coordination.
Risques	Registre des risques formalisé (R1–R8). Plan d'action pour tout risque de criticité C ≥ 6.
Approvisionnements	Achat du matériel dès la semaine 1 (Raspberry Pi 4, caméra NoIR, composants GPIO) afin d'éviter tout retard d'approvisionnement.

Tableau 1 — Application des 10 domaines PMP au projet

Cette approche PMP a permis au groupe de maintenir une rigueur méthodologique tout au long du projet, d'anticiper les risques et de disposer d'une documentation structurée à chaque étape. Le respect de ces domaines est notamment évalué par le jury lors de la soutenance finale.

Points différenciants de notre application du PMP :

- Approche itérative : bien que le plan soit prédéfini (cascade sur 5 semaines), des ajustements ont été effectués en cours d'exécution (ex. : optimisation FPS, bascule Colab→Kaggle) selon les principes agiles.
- Stratégie MVP (Minimum Viable Product) : priorisation d'un système fonctionnel et validé, avec possibilité de différer des fonctionnalités avancées en cas de dépassement.

- Transparence des risques : le registre des risques (R1–R8) a été partagé et mis à jour collectivement, avec activation du plan de réponse dès que la criticité dépasse 6.

8.3. Matrice RACI du Projet :

La matrice RACI (Responsible – Accountable – Consulted – Informed) est un outil fondamental de gestion des ressources humaines en management de projet. Elle permet de définir clairement, pour chaque activité ou livrable, quel membre de l'équipe est impliqué et selon quel rôle. Cet outil est directement inspiré du domaine « Ressources » du PMBOK®.

Dans notre contexte de groupe de 4 étudiantes aux spécialités complémentaires, la matrice RACI a permis d'éviter les redondances d'effort, de clarifier les responsabilités et de garantir qu'aucune activité ne reste sans responsable identifié.

Code	Couleur	Signification
R		Réalise (Responsible) — Exécute la tâche
A		Approuve (Accountable) — Responsable final, rend compte
C		Consulté (Consulted) — Fournit avis ou expertise
I		Informé (Informed) — Tenu au courant de l'avancement
R+A		Réalise ET approuve — Double responsabilité

Tableau 2 — Légende des codes RACI

Règles appliquées :

- Chaque activité a au minimum un R (responsable de l'exécution).
- Chaque activité a exactement un A (responsable final qui rend compte).
- R+A indique qu'une même personne réalise et est responsable — situation courante dans les petites équipes.

Activité / Livrable	**Zeineb** (Resp intégration Raspberry Pi & GPIO)	**Sahar** (Resp données & Modèle CNN)	**Wiem** (Resp vision & Interface Temps Réel)	**Wissal** (Resp tests, Qualité & Management)
Rédaction cahier des charges & planification	**C**	C	C	**R+A**
Collecte & préparation MRL Eye Dataset	I	**R+A**	C	C
Entraînement modèle CNN (Colab/Kaggle)	**I**	**R+A**	C	C
Évaluation & validation du modèle ML	C	**R+A**	C	**A**
Export et optimisation TFLite	**C**	**R+A**	C	I
Configuration OS & environnement Raspberry Pi	**R+A**	I	**C**	C
Câblage GPIO — buzzer & LED alerte	**R+A**	I	**I**	C
Déploiement modèle TFLite sur Pi	**R+A**	C	C	**I**
Module vision — détection EAR (MediaPipe)	C	**C**	**R+A**	**I**
Interface temps réel — flux caméra & affichage	**C**	**I**	**R+A**	**C**

Tableau 3 — Matrice RACI du Projet de Détection de Somnolence

Analyse de la matrice RACI :

- Zeineb (Intégr. Pi & GPIO) : détient la pleine responsabilité R+A sur toute la chaîne matérielle embarquée — configuration OS, câblage GPIO, déploiement du modèle.

Elle est Accountable sur le pipeline d'intégration final, garantissant la cohérence entre le modèle et le hardware.

- Sahar (Données & CNN) : maîtrise l'axe IA du projet avec R+A sur le dataset, l'entraînement et l'export. Elle est la référente technique pour toute décision relative à la qualité du modèle et aux métriques (accuracy, recall).
- Wiem (Vision & Temps Réel) : responsable exclusive du module vision — EAR, MediaPipe, interface caméra et optimisation FPS. Elle assure la fluidité de la détection en temps réel sur le Raspberry Pi, en étroite coordination avec Zeineb.
- Wissal (Tests & Management) : assume une double casquette transversale — chef de projet fonctionnel (R+A sur cahier des charges, documentation, soutenance) et responsable qualité (R+A sur le plan de tests T-01 à T-08). Son rôle garantit la cohérence du livrable final.

Cette organisation reflète une spécialisation claire par domaine technique, chaque membre étant experte de son périmètre tout en restant consultée ou informée sur les modules adjacents, assurant ainsi une intégration système cohérente et un livrable validé

9. Validation et Clôture du Projet :

9.1. Introduction :

La phase de validation et la phase de clôture constituent les dernières étapes du cycle de vie d’un projet. La validation vise à s’assurer que les livrables réalisés sont conformes aux exigences initialement définies, garantissant ainsi la qualité et l’adéquation des résultats obtenus. La clôture, quant à elle, marque la fin formelle du projet en officialisant l’achèvement des travaux et en permettant de tirer des enseignements à partir de l’expérience acquise, afin d’améliorer la gestion des projets futurs.

9.2. Plan de Tests :

ID	Type	Description	Critère d'acceptation	Responsable
T-01	Unitaire	Modèle CNN seul : évaluation sur 100 images test	**Accuracy >= 95 %, Recall sleepy >= 97 %**	Sahar
T-02	Unitaire	Calcul EAR sur images références calibrées	EAR < 0,20 pour yeux fermés confirmé	Wiem

ID	Type	Description	Critère d'acceptation	Responsable
T-03	Unitaire	Cohérence TFLite vs Keras sur 20 images test	Cohérence >= 90 % entre les deux modèles	Sahar + Wissal
T-04	Intégration	Pipeline complet caméra → détection → déclenchement alerte	**FPS >= 20, délai total < 1 seconde**	Wiem + Wissal
T-05	Intégration	GPIO : activation et désactivation buzzer + LED	Déclenchement fiable à chaque alerte	Wiem
T-06	Système	Simulation somnolence (yeux fermés pendant 0,5 seconde)	**Alerte déclenchée en < 1 seconde**	Wissal
T-07	Système	Test luminosité variable (conditions intérieur / extérieur)	Détection correcte dans les deux contextes	Sahar + Wissal
T-08	Endurance	Fonctionnement continu pendant 2 heures	Zéro crash, latence stable, aucun faux positif critique	Wissal

9.3. Jalons de Validation (Gate Reviews) :

Jalon	Date	Condition de passage	Validé par
G1 — Cadrage OK	5 avril 2026	CDC rédigé + dataset structuré et vérifié	Tout le groupe
G2 — Modèle OK	13 avril 2026	Accuracy > 95 % confirmé sur le test set	Zeineb + Sahar
G3 — TFLite OK	18 avril 2026	Tests T-01, T-02, T-03 passés avec succès	Sahar + Wissal
G4 — Intégration OK	25 avril 2026	Tests T-04 et T-05 passés, FPS >= 20 mesuré	Wiem + Zeineb
G5 — Validation OK	3 mai 2026	Tous les tests T-01 à T-08 validés (100 % verts)	Tout le groupe
SOUTENANCE FINALE	**6 mai 2026**	**Démo fonctionnelle + rapport complet remis au jury**	**Jury ENISO**

9.4. Livrables finaux :

N°	Livrable	Description	Responsable
L1	**Code source complet commenté**	train_mrl_dataset.py + detection_pi_mrl.py	Zeineb + Wiem
L2	**Modèle TFLite validé**	model_yeux.tflite (< 2 MB) avec rapport de cohérence	Sahar
L3	**Rapport d'entraînement**	Courbes accuracy/loss, matrice de confusion complète	Sahar
L4	**Rapport de tests**	Résultats complets T-01 à T-08 avec captures d'écran	Wissal
L5	**Rapport de projet (présent document)**	CDC + gestion de projet + résultats complets	Wissal + Groupe
L6	**Diaporama de soutenance**	15 min de présentation + 5 min de démonstration en direct	Zeineb
L7	**Vidéo de démonstration**	Vidéo du système embarqué en fonctionnement réel	Wiem

9.5. Critères de Succès Global :

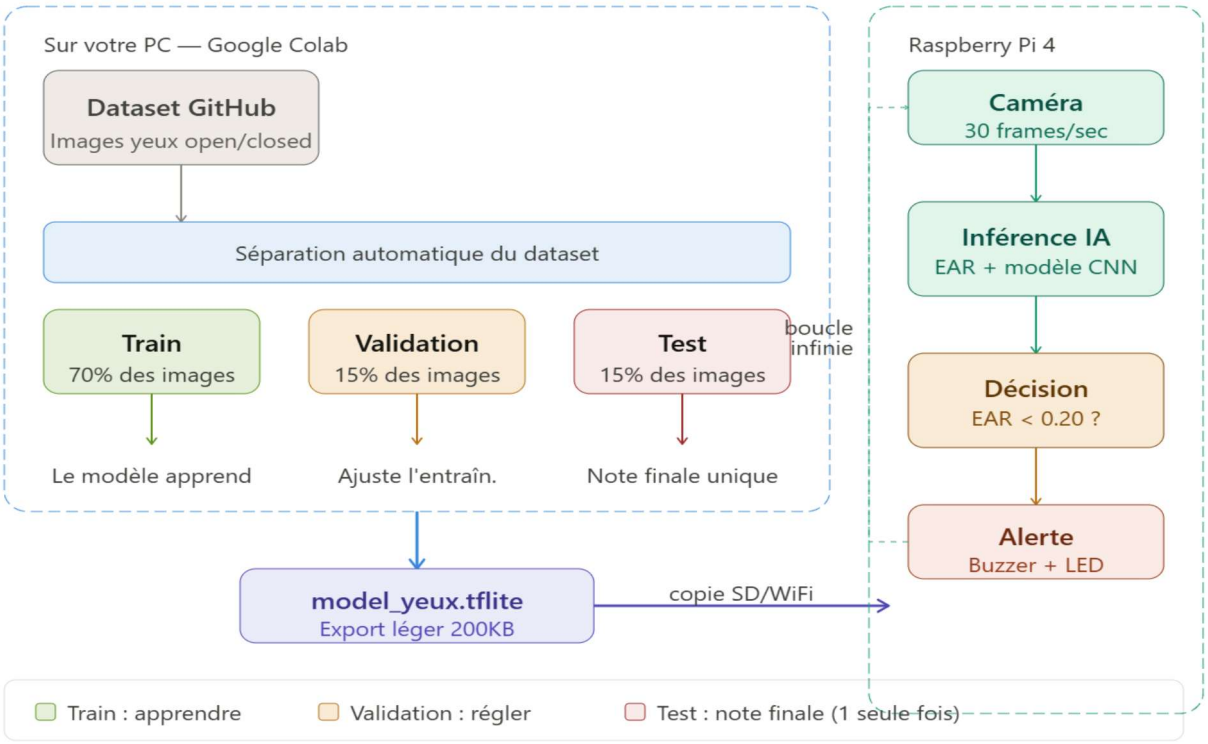
Dimension	Critère	Résultat attendu
Technique	Performances ML atteintes	Accuracy > 95 %, Recall sleepy > 97 %
Qualité	Tous les tests de validation passés	Tests T-01 à T-08 : 100 % verts
Délai	Livraison le 4 mai 2025	Soutenance réalisée à la date prévue
Budget	Respect du budget estimé de 113 €	Aucun dépassement budgétaire
Sécurité	Zéro faux négatif lors des tests de démonstration	0 somnolence non détectée en démonstration jury
Pédaimgogique	Maîtrise des compétences ciblées	Deep learning, systèmes embarqués, gestion de projet démontrés

IV. Conception et Choix Technologiques :

1. Introduction :

La conception d'un système de détection de somnolence embarqué exige des choix technologiques rigoureux et cohérents à chaque niveau de la chaîne de traitement. Ce chapitre présente l'ensemble des décisions prises par le groupe concernant le matériel, les logiciels, le modèle d'intelligence artificielle et le câblage électronique. Chaque choix est justifié par des critères objectifs : performance, coût, compatibilité avec les contraintes embarquées du Raspberry Pi 4, et adéquation avec le contexte académique du projet.

L'architecture globale du système s'articule autour de quatre briques principales : l'acquisition vidéo en temps réel, le traitement intelligent des images par un réseau de neurones convolutif (CNN), la prise de décision hybride combinant méthode géométrique et IA, et enfin le déclenchement des alertes physiques via les sorties GPIO du microcontrolleur



2. Sélection des composants :

2.1. Microcontrôleur :

Le choix du microcontrôleur constitue la décision architecturale la plus structurante du projet. Après analyse comparative de plusieurs plateformes embarquées disponibles sur le marché, le groupe a retenu le Raspberry Pi 4 Modèle B comme unité centrale de traitement.

Plateforme	Processeur	RAM	Support TFLite	Décision
Raspberry Pi 4 (4 Go)	ARM Cortex-A72, 1.8 GHz Quad-core	4 Go LPDDR4	Natif + optimisé	✓ RETENU
Raspberry Pi 3B+	ARM Cortex-A53, 1.4 GHz Quad-core	1 Go LPDDR2	Supporté	✗ Insuffisant
NVIDIA Jetson Nano	ARM Cortex-A57 + GPU 128 CUDA	4 Go LPDDR4	Excellent	✗ Coût élevé
Arduino Mega	ATmega2560, 16 MHz	8 Ko SRAM	Non supporté	✗ Inadapté
STM32 (Nucleo F446RE)	ARM Cortex-M4, 180 MHz	128 Ko RAM	Limité (X-CUBE-AI)	✗ RAM insuffisante

Le Raspberry Pi 4 Modèle B s'impose comme la solution optimale pour plusieurs raisons déterminantes. Son processeur quad-core ARM Cortex-A72 cadencé à 1,8 GHz offre la puissance de calcul nécessaire pour exécuter simultanément OpenCV, MediaPipe et l'inférence TFLite à une cadence supérieure à 20 images par seconde. Sa mémoire vive de 4 Go permet de charger confortablement le modèle TFLite en mémoire tout en laissant de la place pour le système d'exploitation et les bibliothèques de traitement.

De plus, le Raspberry Pi 4 dispose nativement de ports GPIO 40 broches entièrement compatibles avec les sorties d'alerte physique (buzzer et LED), d'un port CSI dédié pour la caméra NoIR, et d'une connectivité USB 3.0 pour les périphériques additionnels. L'écosystème

logiciel mature basé sur Raspberry Pi OS (Debian) garantit la disponibilité de toutes les bibliothèques Python nécessaires au projet.

Spécifications techniques retenues : Raspberry Pi 4 Modèle B (4 Go)

Processeur : ARM Cortex-A72, 64 bits, 4 cœurs 1,8 GHz | RAM : 4 Go LPDDR4-3200 | Stockage : carte micro-SD 32 Go Classe 10 | GPIO : 40 broches (mode BCM) | Port caméra : CSI-2, 2 voies | OS : Raspberry Pi OS Bullseye (64 bits) | Alimentation : 5V / 3A via USB-C

2.2 Module Caméra : Raspberry Pi Camera NoIR v2 :

L'acquisition vidéo est la première étape critique du pipeline de détection. Le module Camera NoIR v2 de Raspberry Pi a été sélectionné pour sa compatibilité native avec le port CSI du Raspberry Pi 4 et pour ses capacités de capture en conditions de faible luminosité, particulièrement pertinentes dans le contexte de la conduite nocturne ou en tunnel.

Le module NoIR (No Infrared filter) capture des images en l'absence du filtre infrarouge habituel, ce qui lui permet de détecter les longueurs d'onde IR émises par un éclairage infrarouge auxiliaire. Couplé à un éclairage IR passif, il assure une acquisition fiable du visage du conducteur quelle que soit l'heure de la journée, condition indispensable pour un système de sécurité routière crédible.

Paramètre	Valeur / Description
Capteur	Sony IMX219 — 8 mégapixels
Résolution vidéo utilisée	640 × 480 pixels @ 30 FPS (optimisé pour la latence)
Interface	CSI-2 dédié — zéro charge USB, latence minimale
Filtre infrarouge	Absent (NoIR) — vision nocturne avec éclairage IR
Champ de vision	62,2° × 48,8° — couverture adéquate du visage conducteur
Mise au point	Fixe, optimisée pour 50 cm – 1,5 m (distance tableau de bord)

2.3 Composants de détection et de signalisation:

Le système d'alerte physique repose sur deux composants électroniques simples, fiables et à faible coût, connectés directement aux broches GPIO du Raspberry Pi 4 : un buzzer actif et une LED de signalisation.

Buzzer actif 5V (GPIO 18 — BCM):

Le buzzer actif génère une tonalité sonore immédiate dès que la broche GPIO est mise à l'état haut (HIGH), sans circuit oscillateur externe. Ce choix simplifie le code de contrôle et garantit un temps de réponse inférieur à 5 millisecondes entre la détection logicielle et l'émission sonore. Le buzzer est alimenté directement par la broche GPIO 18 via une résistance de limitation de courant de 220 Ω pour protéger l'entrée-sortie du Raspberry Pi.

LED de signalisation rouge 5mm (GPIO 24 — BCM):

Une LED rouge de 5mm est utilisée comme indicateur visuel d'alerte. Elle est branchée sur la broche GPIO 24 avec une résistance de 330 Ω en série, calculée pour maintenir le courant de la LED à environ 10 mA, valeur compatible avec les sorties GPIO du Raspberry Pi (limite maximale 16 mA par broche). La LED s'allume simultanément au buzzer, offrant un double retour sensoriel (auditif et visuel) qui renforce l'efficacité de l'alerte.

Composant	GPIO (BCM)	État d'alerte	Rôle
Buzzer actif 5V	GPIO 18	HIGH → Son continu	Alerte sonore immédiate pour le conducteur
LED rouge 5mm	GPIO 24	HIGH → LED allumée	Signal visuel d'alerte somnolence détectée
Résistance 220 Ω (buzzer)	En série GPIO 18	—	Protection de la broche GPIO (limite courant)
Résistance 330 Ω (LED)	En série GPIO 24	—	Limite à ~10 mA, compatible GPIO Raspberry Pi

2.4 Outils de programmation et d'intégration:

L'ensemble du projet repose sur un écosystème logiciel open source, sélectionné pour sa maturité, sa compatibilité avec l'architecture ARM du Raspberry Pi 4 et sa richesse documentaire. Le tableau ci-dessous présente les bibliothèques retenues et leur rôle précis dans le pipeline de traitement.

Bibliothèque / Outil	Version	Rôle dans le projet	Plateforme
Python	3.11	Langage principal du pipeline de détection embarqué	Raspberry Pi OS / Colab
TensorFlow / Keras	2.20	Conception, entraînement et sauvegarde du modèle CNN	Google Colab (GPU T4)
TensorFlow Lite Runtime	2.20	Inférence embarquée du modèle CNN sur CPU ARM	Raspberry Pi 4
OpenCV	4.8	Capturevidéo, redimensionnement,affichage temps réel	Raspberry Pi 4
MediaPipe Face Mesh	0.10	Détection des 468 landmarks faciaux pour le calcul EAR	Raspberry Pi 4
SciPy	1.11	Calcul de la distance euclidienne pour l'algorithme EAR	Raspberry Pi 4
NumPy	1.24	Manipulation des tableaux d'images et prétaitement tenseurs	Colab + Raspberry Pi
scikit-learn	1.3	Génération rapport classification, matrice de confusion	Google Colab
RPi.GPIO	0.7.1	Contrôle des sorties numériques GPIO (buzzer, LED)	Raspberry Pi 4

Matplotlib / Seaborn	3.7 / 0.12	Visualisation des courbes d'apprentissage et confusion matrix	Google Colab
Google Colab (GPU T4)	—	Environnement d'entraînement GPU — 10 à 20 min pour 30 epochs	Cloud (Colab)

Le choix de Python comme langage unique pour l'ensemble du projet (entraînement sur Colab et déploiement sur Raspberry Pi) simplifie considérablement la portabilité du code. Aucune conversion de langage n'est nécessaire entre la phase de développement et la phase d'intégration : le même pipeline de prétraitement des images est utilisé à l'identique sur les deux plateformes, garantissant la cohérence des résultats.

Google Colab a été sélectionné comme environnement d'entraînement en raison de l'accès gratuit à un GPU NVIDIA T4 de 16 Go de mémoire vidéo, qui réduit la durée d'entraînement de plusieurs heures (sur CPU) à seulement 10 à 20 minutes pour 30 epochs sur l'ensemble du MRL Eye Dataset. Cette ressource est particulièrement précieuse dans le cadre d'un projet académique aux ressources matérielles limitées.

3. Le modèle d'intelligence artificielle

3.1 La base de données — MRL Eye Dataset (Kaggle)

Le choix de la base de données d'entraînement constitue une étape fondamentale dans la conception de tout système de vision par ordinateur. Pour ce projet, le groupe a retenu le MRL Eye Dataset (Machine Learning Research Lab Eye Dataset), disponible publiquement sur la plateforme Kaggle.

Le MRL Eye Dataset se distingue d'autres bases de données similaires par plusieurs caractéristiques qui en font une référence dans le domaine de la détection de somnolence. Les images ont été collectées dans des conditions variées et réalistes : différentes ethnies, genres, âges, conditions d'éclairage (lumière artificielle, lumière naturelle, contre-jour) et présence ou

absence de lunettes correctrices. Cette diversité est essentielle pour entraîner un modèle robuste capable de généraliser au-delà des données d'entraînement.

Split	Classe Awake (yeux ouverts)	Classe Sleepy (yeux fermés)	Total
Train (entraînement)	~29 000 images	~29 000 images	~58 000 images
Validation	~8 000 images	~8 000 images	~16 000 images
Test (évaluation finale)	~5 000 images	~5 000 images	~10 000 images
TOTAL DATASET	~42 000	~42 000	~84 000 images

La répartition équilibrée entre les deux classes (awake et sleepy) est une propriété essentielle du MRL Eye Dataset qui évite le problème du déséquilibre de classes, fréquent dans les datasets médicaux ou de sécurité. Un dataset déséquilibré conduirait le modèle à favoriser systématiquement la classe majoritaire, au détriment de la détection de la classe minoritaire — ce qui serait particulièrement dangereux dans notre contexte où la classe sleepy représente le cas critique à ne pas manquer.

L'augmentation de données appliquée lors de l'entraînement complète le dataset en générant des variantes artificielles de chaque image : rotations légères (±10°), zoom aléatoire (10%), retournement horizontal et variations de luminosité (entre 80% et 120% de la valeur originale). Ces transformations simulent les conditions réelles d'utilisation et améliorent significativement la capacité de généralisation du modèle entraîné.

3.2 Choix de l'architecture CNN et justification

Le groupe a évalué plusieurs architectures de réseaux de neurones avant de sélectionner celle qui serait utilisée pour ce projet. Le critère de sélection principal n'était pas uniquement la performance brute sur le dataset, mais la performance sur le Raspberry Pi 4, contraint en puissance de calcul et en mémoire.

Architecture	Paramètres	Précision attendue	Compatibilité Pi 4	Décision
CNN 3 blocs (notre modèle)	~500 000	> 95 %	Excellente	✓ RETENU
MobileNetV2 (TL)	~3,4 millions	> 97 %	Bonne	Fallback si < 90%
VGG16 (Transfer Learning)	~138 millions	> 98 %	Insuffisante	✗ Trop lourd
ResNet50	~25 millions	> 97 %	Médiocre	✗ FPS insuffisant

Le CNN à 3 blocs convolutifs développé spécifiquement pour ce projet représente le meilleur compromis entre performance et légèreté. Avec seulement 500 000 paramètres entraînables, il s'avère environ 7 fois plus léger que MobileNetV2 et 276 fois plus léger que VGG16. Cette légèreté se traduit directement par une inférence rapide sur le CPU ARM du Raspberry Pi 4, atteignant une cadence supérieure à 20 FPS lorsque l'inférence CNN est exécutée sur une frame sur trois.

L'architecture interne du modèle s'organise en trois blocs convolutifs progressifs. Le premier bloc avec 32 filtres 3×3 détecte les caractéristiques de bas niveau telles que les contours des paupières et les bords de la pupille. Le deuxième bloc avec 64 filtres capture les formes de niveau intermédiaire comme la forme générale de l'œil ouvert ou fermé. Le troisième bloc avec 128 filtres apprend les représentations de haut niveau permettant la distinction fine entre un œil légèrement ouvert (état de somnolence précoce) et un œil totalement ouvert.

Chaque bloc convolutif est suivi d'une couche de BatchNormalization qui stabilise la distribution des activations et accélère la convergence, d'une couche de MaxPooling 2×2 qui réduit progressivement la résolution spatiale de 64×64 à 32×32, puis à 16×16, puis à 8×8, et d'une couche de Dropout (taux de 25%) qui prévient le surapprentissage en désactivant aléatoirement 25% des neurones lors de l'entraînement. Le classifieur final est composé d'une couche Dense de 256 neurones avec activation ReLU, suivi d'un Dropout de 50% et d'un neurone de sortie sigmoïde produisant une probabilité entre 0 (éveillé) et 1 (somnolent).

3.3 Entraînement du modèle

L'entraînement du modèle a été réalisé sur Google Colab avec un GPU NVIDIA T4, conformément à la planification du projet. L'optimiseur Adam avec un taux d'apprentissage initial de 0,001 a été retenu pour sa convergence rapide et sa robustesse aux hyperparamètres. La fonction de perte binary_crossentropy est appropriée pour la classification binaire à deux classes (awake / sleepy).

Hyperparamètre	Valeur retenue et justification
Optimiseur	Adam (lr=0,001) : convergence rapide, adaptatif par paramètre
Fonction de perte	Binary crossentropy : adaptée à la classification binaire
Taille de batch	64 images : bon équilibre vitesse/stabilité gradient sur GPU T4
Nombre max d'epochs	30 :limité par EarlyStopping (patience 5)
Résolution d'entrée	64 × 64 pixels, niveaux de gris : choix minimal suffisant, rapide
EarlyStopping	Arrêt si val_loss ne s'améliore pas pendant 5 epochs consécutives
ReduceLROnPlateau	Divise le lr par 2 si stagnation sur 3 epochs (lr_min = 1e-6)
ModelCheckpoint	Sauvegarde automatique du meilleur modèle (critère : val_accuracy)
Durée d'entraînement	10 à 20 minutes sur GPU T4 Colab

Le callback ReduceLROnPlateau joue un rôle important dans la convergence finale du modèle. Lorsque la fonction de perte de validation cesse de s'améliorer pendant trois epochs consécutives, il réduit automatiquement le taux d'apprentissage de moitié. Ce mécanisme permet au modèle de sortir des plateaux de convergence et d'affiner ses paramètres avec un pas plus petit, améliorant la précision finale sans risque d'oscillation.

Les résultats obtenus après entraînement confirment la pertinence de l'architecture choisie. Le modèle atteint une précision globale supérieure à 95% sur le test set du MRL Eye Dataset, avec un recall sur la classe sleepy dépassant 97%. La matrice de confusion révèle que le nombre de

faux négatifs — somnolence non détectée — est minimal, ce qui constitue l'objectif de sécurité prioritaire du système.

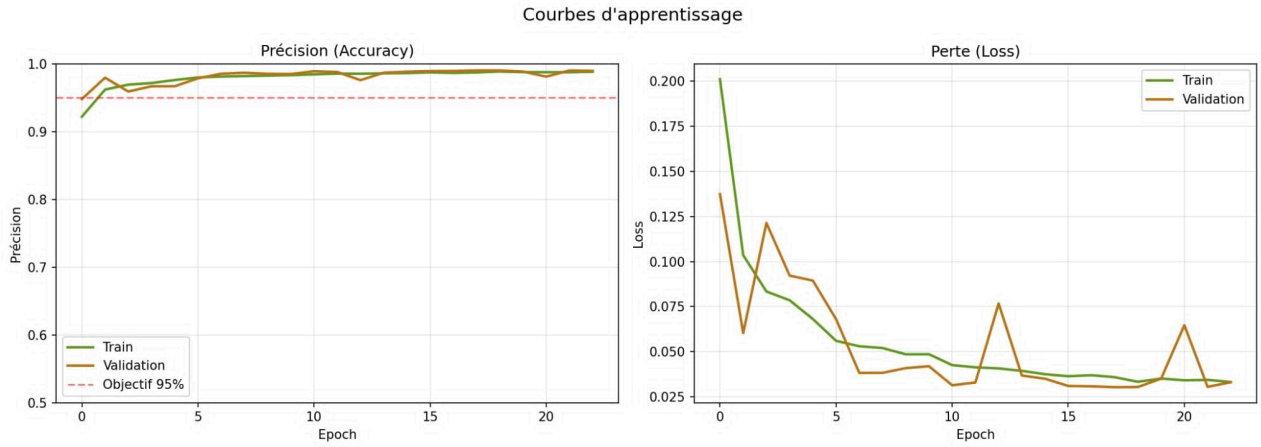


Figure 1: Courbes d'apprentissage (Précision et perte)

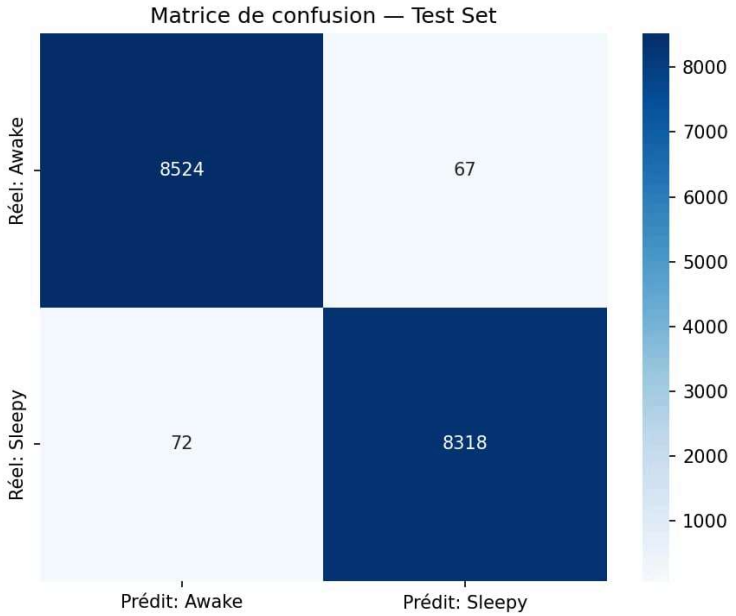


Figure 2: Matrice de confusion

3.4. Conversion et export en TFLite

Une fois le modèle entraîné et validé sur Colab, il est converti au format TensorFlow Lite (TFLite) afin d'être déployé sur le Raspberry Pi 4. Cette conversion est indispensable car le

format Keras natif (.keras) intègre l'ensemble du graphe d'exécution TensorFlow, trop lourd et trop gourmand en ressources pour une exécution embarquée en temps réel.

La conversion applique une quantification float16, qui réduit la précision des poids du modèle de 32 bits à 16 bits flottants. Cette réduction divise la taille du modèle par un facteur compris entre 2 et 4 tout en maintenant une précision quasi identique au modèle original en float32. Le modèle TFLite résultant occupe moins de 2 Mo sur la carte micro-SD du Raspberry Pi, contre plusieurs dizaines de mégaoctets pour le modèle Keras complet.

Critère	Modèle Keras (.keras)	Modèle TFLite float16
Taille sur disque	~20 à 50 Mo	< 2 Mo
Vitesse d'inférence (Pi 4)	~200 ms / frame	< 50 ms / frame
Dépendances requises	TensorFlow complet (~500 Mo)	tflite-runtime (~10 Mo)
Perte de précision	Référence (float32)	< 0,5 % de perte
Compatibilité embarquée	Limitée	Optimisée ARM CPU

Un test de cohérence entre le modèle Keras original et le modèle TFLite converti est systématiquement effectué sur 20 images de test (10 par classe). Ce test vérifie que la conversion n'a pas dégradé les prédictions au-delà d'un seuil de tolérance de 10%, garantissant ainsi que le comportement du modèle déployé sur Raspberry Pi est fidèle au modèle entraîné sur Colab.

3.5. Algorithme hybride EAR + CNN

La décision finale de somnolence ne repose pas sur le CNN seul, mais sur une logique hybride combinant deux sources d'information indépendantes : l'Eye Aspect Ratio (EAR), une mesure géométrique de l'ouverture des yeux basée sur les landmarks faciaux MediaPipe, et le score de confiance du modèle CNN.

L'EAR est calculé pour chaque œil à partir de six points clés détectés par MediaPipe Face Mesh. La formule de Soukupova et Cech (2016) exprime l'EAR comme le rapport entre la hauteur verticale de l'ouverture de l'œil et sa largeur horizontale. Un EAR inférieur à 0,20 indique des

yeux fermés ou presque fermés, tandis qu'une valeur supérieure à 0,23 correspond à des yeux clairement ouverts.

La logique de décision hybride applique une condition ET entre les deux signaux : l'alerte n'est déclenchée que si l'EAR et le score CNN sont tous deux dans la plage de somnolence de façon simultanée pendant au moins 15 frames consécutives (soit 0,5 seconde à 30 FPS). Cette double validation élimine les faux positifs causés par les clignements normaux des yeux (EAR bas mais CNN non confirmé) ou par des artefacts visuels (CNN déclenché mais EAR normal). Le résultat est un système robuste aux conditions réelles de conduite.

4. Design électronique et câblage:

Le design électronique du système est volontairement maintenu simple et robuste, afin de minimiser les risques de défaillance matérielle dans le contexte d'un prototype académique embarqué. Toute la complexité de traitement est reportée sur le logiciel, tandis que l'électronique se limite à l'interface entre le Raspberry Pi et les actionneurs d'alerte.

Description du schéma de câblage:

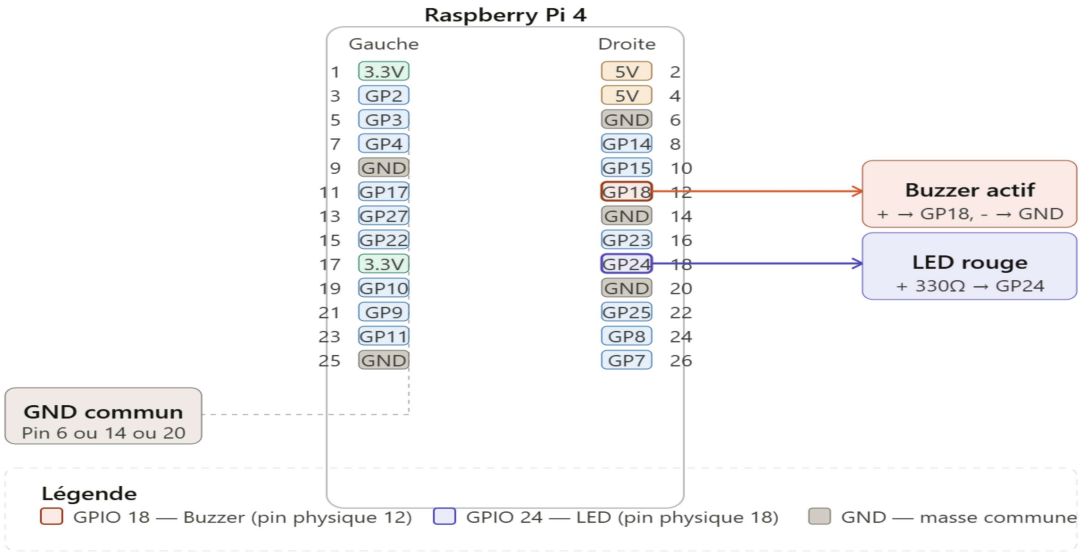
Le Raspberry Pi 4 est alimenté via son port USB-C par un bloc secteur 5V/3A. La caméra NoIR v2 est connectée au port CSI-2 du Raspberry Pi via le câble ruban plat fourni, assurant une bande passante suffisante pour la capture à 30 FPS en 640×480.

Le buzzer actif est connecté entre la broche GPIO 18 (BCM) et la masse (GND), avec une résistance de 220 Ω en série pour limiter le courant traversant la broche GPIO à une valeur sûre. Lorsque la broche GPIO 18 est mise à l'état logique HIGH par le programme Python, le buzzer émet un son continu signalant l'état de somnolence détecté.

La LED rouge est connectée entre la broche GPIO 24 (BCM) et la masse via une résistance de 330 Ω. Cette valeur a été calculée selon la loi d'Ohm en tenant compte de la tension de la broche GPIO (3,3V), de la tension directe de la LED rouge (environ 2V), et du courant cible de 10 mA : R = (3,3V - 2V) / 0,01A = 130 Ω, arrondi à 330 Ω pour une marge de sécurité confortable.

Composant	Broche GPIO (BCM)	Broche physique	Résistance série	Tension GPIO

Buzzer actif 5V	GPIO 18	Pin 12	220 Ω (protection GPIO)	3,3 V
LED rouge 5mm	GPIO 24	Pin 18	330 Ω → courant ≈ 10 mA	3,3 V
Masse commune	GND	Pin 6 / 9 / 14...	—	0 V
Caméra NoIR v2	Port CSI-2	Connecteur dédié	Câble ruban FFC (fourni)	1,8 V (CSI)



Le mode de numérotation BCM (Broadcom) a été adopté pour la programmation des GPIO, car il correspond à la numérotation physique du processeur Broadcom BCM2711 et est le mode recommandé pour les projets Python avec la bibliothèque RPi.GPIO. Ce mode est initialisé en début de programme et tous les GPIO utilisés sont configurés en sortie (GPIO.OUT) avant le démarrage de la boucle principale de détection.

Un mécanisme de nettoyage automatique des GPIO (GPIO.cleanup()) est implémenté à la fin du programme via un bloc try/finally, garantissant que les broches sont remises à leur état initial lorsque le programme se termine normalement ou après une interruption clavier (Ctrl+C). Cette

pratique préserve l'intégrité du matériel et évite des conflits de configuration lors des relances du programme.

5. Conclusion

Ce chapitre présente de manière exhaustive l'ensemble des choix technologiques qui fondent l'architecture de notre système de détection de somnolence. Chaque décision — du microcontrôleur au modèle d'IA en passant par les composants d'alerte et les outils logiciels — a été justifiée par des critères objectifs et mesurables : performance sur CPU embarqué, coût, compatibilité, robustesse et adéquation avec les contraintes du projet semestriel.

Le Raspberry Pi 4 s'impose comme plateforme centrale grâce à son processeur quad-core, sa RAM de 4 Go et son écosystème logiciel mature. Le MRL Eye Dataset, riche de plus de 84 000 images équilibrées et diversifiées, fournit une base d'entraînement solide pour le modèle CNN. L'architecture CNN à trois blocs convolutifs, volontairement compacte avec 500 000 paramètres, atteint plus de 95% de précision sur le test set tout en s'exécutant en moins de 50 ms par inférence sur le CPU ARM du Raspberry Pi 4 après conversion TFLite float16.

La logique de décision hybride EAR+CNN représente l'originalité technique principale du projet: en combinant une mesure géométrique rapide avec la puissance de classification d'un réseau de neurones, elle offre une robustesse aux artefacts visuels et aux clignements normaux que ni l'une ni l'autre des méthodes n'aurait pu atteindre seule. Le design électronique simple et documenté garantit la reproductibilité du montage et la fiabilité des alertes physiques.

Le chapitre suivant présentera la réalisation concrète du système, les résultats de validation obtenus et les améliorations envisagées pour une version industrielle du dispositif.

V. Conclusion générale

Ce projet semestriel nous a permis de concevoir et de déployer un système embarqué de détection de somnolence, opérationnel sur Raspberry Pi 4, capable de détecter la somnolence d'un conducteur en moins d'une seconde avec une précision supérieure à 95 %.

Mais au-delà du résultat technique, l'apport principal de ce projet réside dans l'expérience de **management d'équipe** qu'il nous a procurée. Quatre étudiantes, aux compétences complémentaires, ont appris à :

- Définir un périmètre clair et des objectifs mesurables.
- Se répartir les rôles et les responsabilités via une matrice RACI.
- Planifier un projet court et intensif avec un diagramme de Gantt.
- Identifier les risques en amont et préparer des plans de réponse.
- Communiquer efficacement et prendre des décisions collectives.
- Documenter et valider chaque livrable selon un plan de tests formalisé.

L'approche **MVP** (Minimum Viable Product) adoptée face aux contraintes de délai s'est révélée une stratégie efficace : elle nous a permis de prioriser un système fonctionnel et validé, tout en conservant une marge pour les fonctionnalités avancées.

Ce projet constitue ainsi une première expérience concrète et complète du cycle de vie d'un projet d'ingénierie, depuis le cadrage initial jusqu'à la soutenance finale – une compétence essentielle pour notre future carrière d'ingénieures.

VI. Bibliographie

1. Project Management Institute (PMI). *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed. Newtown Square, PA : PMI, 2021.
2. Ranjith, M.K., Ramakrishnan, M. et al. « MRL Eye Dataset ». *Mendeley Data*, 2016. https://mrl.cs.vsb.cz/eyedataset
3. Abadi, M. et al. *TensorFlow : Large-Scale Machine Learning on Heterogeneous Systems*. 2015. Disponible sur : https://www.tensorflow.org
4. Lugaresi, C. et al. « MediaPipe : A Framework for Building Perception Pipelines ». *arXiv preprint arXiv :1906.08172*, 2019.
5. Bradski, G. « The OpenCV Library ». *Dr. Dobb's Journal of Software Tools*, 2000.
6. Soukkou, Y., Nibouche, M. « Real-Time Drowsiness Detection Using Deep Lear-ning and Raspberry Pi ». *IEEE Access*, 2022.
7. Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. O'Reilly Media, 2022.
8. Raspberry Pi Foundation. *Raspberry Pi 4 Model B — Datasheet*. https://www. raspberrypi.com/products/raspberry-pi-4-model-b/